

或

```
SELECT Sno
FROM SC
WHERE Cno='81001' AND (Grade < 60 OR Grade IS NULL);
```

3.6 视图

视图是关系数据库系统提供给用户以多种角度观察数据库中数据的重要机制。

视图是从一个或几个基本表(或视图)导出的表。它与基本表不同,是一个虚表。数据库中只存放视图的定义,而不存放视图对应的数据,这些数据仍存放在原来的基本表中。所以一旦基本表中的数据发生变化,从视图中查询出的数据也就随之改变了。从这个意义上讲,视图就像一个窗口,透过它可以看到数据库中自己感兴趣的数据及其变化。

视图一经定义,就可以和基本表一样被查询、被删除。也可以在一个视图之上再定义新的视图,但对视图的更新(增、删、改)操作则有一定的限制。

本节讨论视图的定义、操作及作用。

3.6.1 定义视图

1. 建立视图

SQL 语言用 CREATE VIEW 命令建立视图,其一般语法如下:

```
CREATE VIEW <视图名>[( <列名>[, <列名>]...)]
AS <子查询>
[WITH CHECK OPTION];
```

其中,子查询可以是任意的 SELECT 语句,是否包含 ORDER BY 子句和 DISTINCT 短语取决于具体系统的实现。

WITH CHECK OPTION 表示对视图进行 UPDATE、INSERT 和 DELETE 操作时,要保证更新、插入或删除的行满足视图定义中的谓词条件(即子查询中的条件表达式)。

组成视图的属性列名或者全部省略,或者全部指定,没有第三种选择。如果省略了视图的各个属性列名,则隐含该视图由子查询中 SELECT 子句目标列中的诸字段组成。但在下列三种情况下必须明确指定组成视图的所有列名:

- ① 某个目标列不是单纯的属性名,而是聚集函数或列表表达式。
- ② 多表连接时选出了几个同名列作为视图的字段。
- ③ 需要在视图中为某个列启用新的更合适的名字。

[例 3.86] 建立信息管理与信息系统专业学生的视图。

```
CREATE VIEW IS_Student
AS
SELECT Sno,Sname,Ssex,Sbirthdate, Smajor
FROM Student
WHERE Smajor='信息管理与信息系统';
```

本例中省略了视图 IS_Student 的列名,表示隐含由子查询中 SELECT 子句的 5 个列名组成。

关系数据库管理系统执行 CREATE VIEW 语句的结果只是把视图的定义存入数据字典,并不执行其中的 SELECT 语句。只有在对视图查询时,才按视图的定义从基本表中将数据查出。

[例 3.87] 建立信息管理与信息系统专业学生的视图,并要求进行插入、修改和删除操作时,仍需保证该视图只有信息管理与信息系统专业的学生。

```
CREATE VIEW IS_Student
AS
SELECT Sno,Sname,Ssex,Sbirthdate, Smajor
FROM Student
WHERE Smajor='信息管理与信息系统'
WITH CHECK OPTION;
```

由于在定义 IS_Student 视图时加上了 WITH CHECK OPTION 子句,以后对该视图进行插入、修改和删除操作时,关系数据库管理系统会自动检查 Smajor='信息管理与信息系统'的条件。

例如:

```
INSERT INTO IS_Student values ('20180010','贾明','男','2001-11-1','信息管理与信息系统');
```

插入成功。

```
INSERT INTO IS_Student values ('2018011','王伟','男','2003-11-1','计算机科学与技术');
```

插入失败。原因是不符合 WITH CHECK OPTION 的条件。

若一个视图是从单个基本表导出的,并且只是去掉了基本表的某些行和某些列,但保留了主码,则称这类视图为行列子集视图。IS_Student 视图就是一个行列子集视图。

视图不仅可以建立在单个基本表上,也可以建立在多个基本表上。

[例 3.88] 建立信息管理与信息系统专业选修 81001 号课程的学生视图(包括学号、姓名、成绩属性)。

```
CREATE VIEW IS_C1(Sno,Sname,Grade)
AS
```

```
SELECT Student. Sno, Sname, Grade
FROM Student, SC
WHERE Smajor='信息管理与信息系统' AND Student. Sno=SC. Sno AND SC. Cno='81001';
```

由于视图 IS_C1 的属性列取自 Student 表与 SC 表的部分属性列, 所以必须在 CREATE VIEW IS_C1 后面明确说明视图的各个属性列名。

视图不仅可以建立在一个或多个基本表上, 也可以建立在一个或多个已定义好的视图上, 或建立在基本表与视图上。

[例 3.89] 建立信息管理与信息系统专业选修 81001 号课程且成绩在 90 分以上的学生的视图(包括学号、姓名、成绩属性)。

```
CREATE VIEW IS_C2
AS
SELECT Sno, Sname, Grade
FROM IS_C1
WHERE Grade>=90;
```

这里的视图 IS_C2 就是建立在视图 IS_C1 之上的。

定义基本表时, 为了减少数据库中的冗余数据, 表中只存放基本数据, 由基本数据经过各种计算派生出的数据一般是不存储的。由于视图中的数据并不实际存储, 所以定义视图时可以根据应用的需要设置一些派生属性列。这些派生属性由于在基本表中并不实际存在, 所以也称为虚拟列。带虚拟列的视图称为带表达式的视图。

[例 3.90] 将学生的学号、姓名、年龄定义为一个视图。

```
CREATE VIEW S_AGE( Sno, Sname, Sage)
AS
SELECT Sno, Sname, (extract(year from current_date)-extract(year from Sbirthdate))
FROM Student;
```

这里视图 S_AGE 是一个带表达式的视图。视图定义的 AS 子句中 SELECT 语句的学生年龄是通过计算得到的。

还可以用带有聚集函数和 GROUP BY 子句的查询来定义视图, 这种视图称为分组视图。

[例 3.91] 将学生的学号及平均成绩定义为一个视图。

```
CREATE VIEW S_GradeAVG( Sno, Gavg)
AS
SELECT Sno, AVG( Grade)
FROM SC
GROUP BY Sno;
```

由于 AS 子句中 SELECT 语句的目标列平均成绩是通过聚集函数得到的, 所以 CREATE

VIEW 中必须明确定义组成 S_GradeAVG 视图的各个属性列名。S_GradeAVG 是一个分组视图。

[例 3.92] 将 Student 表中所有的女生记录定义为一个视图。

```
CREATE VIEW F_Student(Fsno,Fname,Fsex,Fbirthdate,Fmajor)
AS
SELECT *
FROM Student
WHERE Ssex='女';
```

这里视图 F_Student 是由子查询“SELECT *”建立的。F_Student 视图的属性列与 Student 表的属性列一一对应。如果以后修改了基本表 Student 的结构,则 Student 表与 F_Student 视图的映像关系可能会被破坏,该视图就不能正常工作了。为避免出现这类问题,最好在修改基本表之后删除由该基本表导出的视图,然后重建这个视图。

2. 删除视图

该语句的格式如下:

```
DROP VIEW <视图名>[ CASCADE];
```

视图删除后视图的定义将从数据字典中删除。如果该视图上还导出了其他视图,则使用 CASCADE 级联删除语句把该视图和由它导出的所有视图一起删除。

基本表删除后,由该基本表导出的所有视图均无法使用了,但是视图的定义没有从字典中清除。删除这些视图定义需要显式地使用 DROP VIEW 语句。

[例 3.93] 删除视图 S_AGE 和视图 IS_C1:

```
DROP VIEW S_AGE;          /* 成功执行 */
DROP VIEW IS_C1;          /* 报告错误 */
```

执行 DROP VIEW IS_C1 语句时,由于 IS_C1 视图上还导出了 IS_C2 视图,所以该语句执行时会报告错误,提示视图 IS_C2 依赖于视图 IS_C1。

如果导出视图也确定可以删除,则使用级联删除语句:

```
DROP VIEW IS_C1 CASCADE;
```

执行此语句不仅删除了 IS_C1 视图,还级联删除了由它导出的 IS_C2 视图。

3.6.2 查询视图

视图定义后,用户就可以像对基本表一样对视图进行查询了。

[例 3.94] 在信息管理与信息系统专业学生的视图中,找出年龄小于或等于 20 岁的学生(包括学生的学号和出生日期)。

```
SELECT Sno, Sbirthdate
```

```
FROM IS_Student
WHERE( extract(year from current_date)-extract(year from Sbirthdate) )<= 20;
```

关系数据库管理系统执行对视图的查询时，首先进行有效性检查，检查查询中涉及的表、视图等是否存在。如果存在，则从数据字典中取出视图的定义，把定义中的子查询和用户的查询结合起来，转换成等价的对基本表的查询，然后再执行修正了的查询。这一转换过程称为视图消解(view resolution)。

本例中，先找到视图 IS_Student 的定义如下：

```
CREATE VIEW IS_Student
AS
SELECT Sno,Sname,Ssex,Sbirthdate
FROM Student
WHERE Smajor='信息管理与信息系统'
WITH CHECK OPTION;
```

然后进行视图消解，转换后的查询语句如下：

```
SELECT Sno,Sbirthdate
FROM Student
WHERE Smajor='信息管理与信息系统' AND
      (extract(year from current_date)-extract(year from Sbirthdate))<= 20;
```

[例 3.95] 查询选修 81001 号课程的信息管理与信息系统专业学生。

```
SELECT IS_Student. Sno,Sname
FROM IS_Student,SC
WHERE IS_Student. Sno=SC. Sno AND SC. Cno='81001';
```

本查询涉及视图 IS_Student 和基本表 SC 的连接操作，在用户眼里视图如同基本表一样。

关系数据库管理系统先从数据字典中取出视图 IS_Student 的定义，然后进行视图消解，把上面的查询转换为：

```
SELECT Student. Sno,Sname
FROM Student,SC
WHERE Student. Sno=SC. Sno AND SC. Cno='81001' AND Smajor='信息管理与信息系统';
```

最后，通过这两个基本表的连接来完成用户请求。

在一般情况下，视图查询的转换是直截了当的。但有些情况下，这种转换不能直接进行，查询时就会出现问题，如例 3.96。

[例 3.96] 在例 3.91 中定义的视图 S_GradeAVG 视图中，查询平均成绩在 90 分以上的学生学号和平均成绩。

```
SELECT *  
FROM S_GradeAVG          /* FROM 后面是视图 S_GradeAVG */  
WHERE Gavg>=90;
```

将本例中的查询语句进行视图消解,与定义 S_GradeAVG 视图的子查询结合,转换成下列查询语句:

```
SELECT Sno,AVG(Grade)  
FROM SC  
WHERE AVG(Grade)>=90  
GROUP BY Sno;
```

因为 WHERE 子句中是不能用聚集函数作为条件表达式的,因此执行此修正后的查询将会出现语法错误。正确转换的查询语句如下:

```
SELECT Sno,AVG(Grade)  
FROM SC  
GROUP BY Sno  
HAVING AVG(Grade)>=90;
```

目前多数关系数据库管理系统对行列子集视图的查询均能进行正确转换。但对非行列子集视图的查询(如例 3.96)就不一定能正确转换了,因此这类查询应该直接对基本表进行。

例 3.96 也可以用如下 SQL 语句完成:

```
SELECT *  
FROM (SELECT Sno,AVG(Grade)    /* 子查询生成一个派生表 S_GradeAVG */  
      FROM SC  
      GROUP BY Sno ) AS S_GradeAVG(Sno, Gavg)  
WHERE Gavg>=90;
```

注意: 定义视图后对视图进行查询与基于派生表的查询是有区别的。视图一旦定义,该定义将保存在数据字典中,之后的所有查询都可以直接引用该视图。而派生表只是在语句执行时临时定义,语句执行后该派生表定义即被删除。

3.6.3 更新视图

更新视图是指通过视图来插入、删除和修改数据。由于视图是不实际存储数据的虚表,因此对视图的更新最终要转换为对基本表的更新。像查询视图那样,对视图的更新操作也是通过视图消解,转换为对基本表的更新操作。

为防止用户通过视图对数据进行更新操作时,有意无意地对不属于视图范围内的基本表数进行操作,可在定义视图时加上 WITH CHECK OPTION 子句。这样在视图上更新数据时,关系数据库管理系统会检查视图定义中的条件,若不满足条件则拒绝执行该操作。

[例 3.97] 将信息管理与信息系统专业学生视图 IS_Student 中 20180005 号学生姓名改为“刘新奇”。

```
UPDATE IS_Student
SET Sname='刘新奇'
WHERE Sno='20180005';
```

视图消解后，对视图 IS_Student 的更新语句就转换为对基本表 Student 的更新了：

```
UPDATE Student
SET Sname='刘新奇'
WHERE Sno='20180005' AND Smajor='信息管理与信息系统';
```

[例 3.98] 向信息管理与信息系统专业学生视图 IS_Student 中插入一个新的学生记录 (20180207, 赵新, 男, 2001-7-19)。

```
INSERT INTO IS_Student
VALUES('20180207','赵新','男','2001-7-19','信息管理与信息系统');
```

转换为对基本表的更新：

```
INSERT INTO Student(Sno,Sname,Ssex,Sbirthdate,Smajor)
VALUES('20180207','赵新','男','2001-7-19','信息管理与信息系统');
```

这里系统自动将‘信息管理与信息系统’放入 VALUES 子句。

[例 3.99] 删除信息管理与信息系统专业学生视图 IS_Student 中 20180207 号学生的记录。

```
DELETE FROM IS_Student
WHERE Sno='20180207';
```

转换为对基本表的更新：

```
DELETE FROM Student
WHERE Sno='20180207' AND Smajor='信息管理与信息系统';
```

在关系数据库中，并不是所有的视图都是可更新的，因为有些视图的更新不能唯一地有意义地转换成对相应基本表的更新。

例如，例 3.91 定义的视图 S_GradeAVG 是由学号和平均成绩两个属性列组成的，其中平均成绩一项是由 SC 表中对元组分组后计算平均值得来的。

如果想把视图 S_GradeAVG 中学号为“20180001”学生的平均成绩改成 90 分，SQL 语句如下：

```
UPDATE S_GradeAVG
SET Gavg=90
WHERE Sno='20180001';
```

这里对视图的更新无法转换成对基本表 SC 的更新,因为系统无法修改各科成绩以使 20180001 号学生的平均成绩成为 90。所以 S_GradeAVG 视图是不可更新的。

一般地,行列子集视图是可更新的。除行列子集视图外,有些视图理论上是可更新的,还有些视图从理论上就是不可更新的。

应该指出的是,不可更新的视图与不允许更新的视图是两个不同的概念。前者指理论上已证明其是不可更新的视图;后者指实际系统中不支持其更新,但它本身有可能是可更新的视图。

目前,各个关系数据库管理系统一般都只允许对一个表的行列子集视图进行更新,而且各个系统对视图的更新还有更进一步的规定。由于各系统实现方法上的差异,这些规定也不尽相同。读者一定要参考具体的产品手册,正确写出对视图的更新语句。

3.6.4 视图的作用

视图最终是定义在基本表之上的,对视图的一切操作最终也要转换为对基本表的操作,而且对于非行列子集视图进行查询或更新时还有可能出现问题。既然如此,为什么还要定义视图呢?这是因为合理使用视图能够带来许多好处,所以在实际应用开发中经常使用视图。

1. 视图能够对机密数据提供安全保护

有了视图机制,就可以在设计数据库应用系统时对不同的用户定义不同的视图,使机密数据不出现在不应看到这些数据的用户视图上。这样视图机制就自动提供了对机密数据的安全保护功能。假设 Student 表涉及全校 30 个院系的学生数据,可以在其上定义 30 个视图,每个视图只包含一个院系的学生数据,并只允许每个院系的主任查询和修改本院系的学生视图。

2. 视图对重构数据库提供了一定程度的逻辑独立性

第 1 章中已经介绍过数据的物理独立性与逻辑独立性的概念。数据的物理独立性是指用户的应用程序不依赖于数据库的物理结构。数据的逻辑独立性是指当数据库重构时,如增加新的关系或对原有关系增加新的字段等,用户的应用程序不会受影响。层次数据库和网状数据库一般能较好地支持数据的物理独立性,而对于逻辑独立性则不能完全地支持。

在关系数据库中,数据库的重构往往是不可避免的。重构数据库最常见的是将一个基本表“垂直”地分成多个基本表。例如:将学生关系 Student(Sno, Sname, Ssex, Sbirthdate, Smajor)分为 SX(Sno, Sname, Sbirthdate)和 SY(Sno, Ssex, Smajor)两个基本表。这时,原来的表 Student 为 SX 表和 SY 表自然连接的结果。如果建立一个视图 Student:

```
CREATE VIEW Student(Sno,Sname,Ssex,Sbirthdate,Smajor)
AS
SELECT SX.Sno,SX.Sname,SY.Ssex,SX.Sbirthdate,SY.Smajor
FROM SX,SY
WHERE SX.Sno=SY.Sno;
```


这样尽管数据库的逻辑结构改变了,变为 SX 和 SY 两个基本表,但不必修改应用程序。因为我们可以建立视图 Student,它和用户原来创建的 Student 表一模一样。

视图使用户的外模式保持不变,同时无须修改用户的应用程序就能和原来一样进行数据操作。

当然,视图只能在一定程度上提供数据的逻辑独立性,比如上面讲到,对视图的更新是有条件的,因此应用程序中修改数据的语句可能仍会因基本表结构的改变而需要做相应修改。

3. 视图能够简化用户的操作

视图机制使用户可以将注意力集中在所关心的数据上。通过定义视图使数据库看起来结构更加简单、清晰,并且可以简化用户的数据查询操作。

例如,那些定义了若干张表连接的视图就将表与表之间的连接操作对用户隐蔽起来了。换句话说,用户所做的只是对一个虚表的简单查询,而这个虚表是怎样得来的,用户无须了解。

适当利用视图可以更清晰地表达查询,例如,经常需要执行这样的查询“对每个学生找出其获得最高成绩的课程的课程号”。可以先定义一个视图,求出每个同学获得的最高成绩:

```
CREATE VIEW VMGrade
AS
SELECT Sno, MAX(Grade) Mgrade /* 每个学生获得的最高成绩 */
FROM SC
GROUP BY Sno;
```

然后用如下查询语句完成课程号查询:

```
SELECT SC. Sno, Cno
FROM SC, VMGrade
WHERE SC. Sno = VMGrade. Sno AND SC. Grade = VMGrade. Mgrade;
```

4. 视图使用户能以多种角度看待同一数据

视图机制能使不同的用户以不同的方式看待同一数据,当许多不同类型的用户共享同一个数据库时,这种灵活性是非常重要的。例如,有的教师希望了解学生的平均成绩,有的教师希望了解学生的最高成绩和最低成绩,他们都可以在基本表 SC 上定义自己感兴趣的视图,直接对这些视图查询。

由于上述诸多优点,视图被广泛用于实际应用开发中。

本章小结

关系数据库标准语言 SQL 是“数据库系统概论”课程学习和实验的重点。

SQL 可以分为数据定义、数据查询、数据更新、数据控制 4 大部分。人们有时把数据更新称为数据操纵, 或把数据查询与数据更新合称为数据操纵。

本章系统而详尽地讲解了前面三部分的内容。数据控制中的数据安全性和完整性控制将在第 4 章和第 5 章中讲解。

本章在讲解 SQL 的同时, 进一步讲解了关系数据库系统的基本概念, 使关系数据库的许多概念更加具体、更加丰富。

SQL 的数据查询功能非常丰富, 也是比较复杂的, 读者一定要加强实践练习, 举一反三, 牢固掌握 SQL 语句。

习 题 3

- 试述 SQL 的特点。
- 说明在 DROP TABLE 时, RESTRICT 和 CASCADE 的区别。
- 有两个关系 $S(A, B, C, D)$ 和 $T(C, D, E, F)$, 写出与下列查询等价的 SQL 表达式:
 - $\sigma_{A=10}(S)$ 。
 - $\Pi_{A,B}(S)$ 。
 - $S \bowtie T$ 。
 - $S \bowtie_{S.C=T.C} T$ 。
 - $S \bowtie_{A<E} T$ 。
 - $\Pi_{C,D}(S) \times T$ 。
- 用 SQL 语句建立第 2 章习题 2 第 6 题 SPJ 数据库的 4 个表(S、P、J 和 SPJ), 并针对建立的 4 个表用 SQL 语句完成该习题中的查询。
- 针对第 4 题中的 4 个表 S、P、J 及 SPJ, 试用 SQL 完成以下各项操作:
 - 找出所有供应商的姓名和所在城市。
 - 找出所有零件的名称、颜色、重量。
 - 找出使用供应商 S1 所供应零件的工程代码。
 - 找出工程项目 J2 使用的各种零件的名称及其数量。
 - 找出上海厂商供应的所有零件代码。
 - 找出使用上海产的零件的工程名称。
 - 找出没有使用天津产的零件的工程代码。
 - 把全部红色零件的颜色改成蓝色。
 - 把由 S5 供给 J2 的零件 P6 改为由 S3 供应。
 - 从供应商关系中删除 S2 的记录, 并从供应情况关系中删除相应的记录。
 - 请将 (S2, J6, P4, 200) 插入供应情况关系。
- 什么是基本表? 什么是视图? 两者之间的区别和联系是什么?
- 试述视图的优点。
- 哪类视图是可以更新的? 哪类视图是不可更新的? 各举一例说明。

9. 针对第4题中的4个表,为三建工程项目建立一个供应情况的视图,包括供应商代码(SNO)、零件代码(PNO)、供应数量(QTY)。针对该视图完成下列查询:

① 找出三建工程项目使用的各种零件代码及其数量。

② 找出供应商 S1 供应三建工程情况。

10. 什么是空值? 举例说明。SQL 中如何表示空值? 空值如何参加运算?

第3章实验 SQL 查询与操纵

实验 3.1 数据库定义。理解数据库对象及其作用,使用 SQL 数据定义语言(DDL)语句创建数据库模式和基本表。参考《概论辅导书》准备好实验数据,插入建立的基本表中。

实验 3.2 SQL 基本查询。掌握 SQL 的基本语法和基本用法,使用 SQL 查询语句完成单表查询和连接查询操作。

实验 3.3 SQL 高级查询。使用 SQL 嵌套查询和集合查询语句完成各类复杂查询操作。

实验 3.4 数据更新。使用 SQL 数据操纵语言(DML)语句完成各类更新操作(包括插入数据、修改数据、删除数据)。

实验 3.5 视图。理解视图的作用,掌握视图的创建、使用和删除等基本功能。

实验 3.6 索引。理解和掌握索引的设计、创建、使用和维护等功能,体验索引对于数据量大时查询效率提高的效果。

参考文献 3

[1] BOYCE R, CHAMBERLIN D D, KING W F, et al. Specifying queries as relational expressions[J]. CACM-Communications of the ACM, 1975, 18(11): 621-628.

[2] CHAMBERLIN D D, BOYCE R F. SEQUEL: A structured English query language[C]. Proceedings of ACM SIGMOD Workshop on Data Description, Access and Control, 1974: 249-264.

[3] CHAMBERLIN D D, ASTRAHAN M M, ESWARAN K P, et al. SEQUEL 2: a unified approach to data definition, manipulation and control[J]. IBM Journal of Research and Development, 1976, 20(6): 560-575.

SQL 是 Boyce 等人 1975 年在文献[1]提出的 SQUARE(Specifying Queries as Relational Expressions)语言的基础上发展起来的。文献[2]对 SQUARE 语言的语法进行了修改,形成了 SEQUEL。文献[3]对 SEQUEL 进行了进一步改进,最后形成了 SQL。SEQUEL 最早在关系数据库系统 SYSTEM R 上实现。

[4] REISNER P. Use of psychological experimentation as an aid to development of a query language[J]. TSE-IEEE Transactions on Software Engineering, 1977, 3(3): 218-229.

[5] DATE C J. A critique of the SQL database language[J]. ACM SIGMOD Record, 1984, 14(3): 8-54.

文献[4]和[5]分析了 SQL 的优缺点。

[6] X3H2(ANSI Database Committee). American National Standard Database Language SQL: Working Draft[R]. Document X3H2-85-1, 1984.

[7] ANSI. The Database Language SQL[R]. Document ANSI X3.135, 1986.

[8] GRIFFITHS P P, WADE B W. An authorization mechanism for a relational database system[J]. ACM Trans-

actions on Database Systems, 1976, 1(3): 242-255.

[9] CHAMBERLIN D D. A summary of user experience with the SQL data sublanguage[C]. Proceedings of the International Conference on Databases. Aberdeen, 1980.

[10] DATE C J. The Outer Join[C]. Proceedings of the 2nd International Conference on Database(ICODE-2). Cambridge, 1983.

[11] DAYAL U, BERNSTEIN P A. On the updatability of relational views[C]. Proceedings of the 4th International Conference on Very Large Data Base, 1978: 4368-377.

[12] SHASHA D, BONNET P. Database tuning: principles, experiments, and troubleshooting techniques(part I)[J]. Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data, 2002.

[13] Information technology-database languages-SQL-part2 : foundation (SQL/Foundation): ISO/IEC 9075-2 CORR 1-2019[S]. 5th ed. ISO/IEC, 2019.

本章知识点讲解微视频:



SQL 数据定义



基本 SQL 查询



复杂 SQL 查询



空值

第4章

数据库安全性

在第1章中已经讲到,数据库的特点之一是由数据库管理系统提供统一的数据保护功能来保证数据的安全可靠和正确有效。数据库的数据保护主要包括数据库的安全性和完整性。本章讲解数据库的安全性,第5章将讨论数据库的完整性。

本章导读

数据库的安全性问题和计算机系统的安全性是紧密联系的,计算机系统的安全性问题可大致分为三大类,即技术安全类、管理安全类和政策法规类(详见二维码内容)。



扩展阅读: 计算机系统的三类安全性问题

本章讨论数据库技术安全类问题,即从技术上如何保证数据库系统的安全性。

本章主要介绍有关计算机信息安全和数据库安全标准的内容。从4.2节起讲解数据库安全性控制的主要技术。希望读者理解和掌握通过SQL语句实现自主存取控制权限的授予和收回的方法;掌握创建用户、角色,以及分配权限给角色的方法。要加强实验练习掌握这些方法,还要进一步理解什么是强制存取控制机制,如何利用视图技术、审计技术、数据存储加密和传输加密等技术手段提高数据库的安全性。

本章较难理解的内容是强制存取控制机制中确定主体能否存取客体的存取规则,要理解制定该存取规则的原因。

4.1 数据库安全性概述

数据库的安全性是指保护数据库,以防不合法使用所造成的数据泄露、篡改或破坏。

安全性问题不是数据库系统所独有的,所有计算机系统都存在不安全因素,只是在数据库系统中由于大量数据集中存放,而且为众多用户直接共享,数据成为一个部门、企业乃至一个国家重要的资源,从而使安全性问题更为突出。系统安全保护措施是否有效是数据库系统的主要技术指标之一。